

✓ Competition Introduction:

The Pokémon Fusion Image Competition focused on visual representation learning using convolutional autoencoders. Participants trained on about 18,000 Pokémon fusion images to learn compact latent features. Models were evaluated on image reconstruction, type classification across 170 categories via linear probing, and image generation. The final score combined reconstruction and classification accuracy, rewarding models that balanced compression efficiency with meaningful feature learning under size and runtime limits.

The finalized model architecture and pretrained weights, excluding the training procedure, were submitted to the competition server for evaluation on the hidden test set. The reconstruction plot here is only for format reference, as the actual test evaluation output is more abstract and less visually interpretable.

```
from google.colab import drive
import os
```

```
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Mount to Google Drive & Switch to the dataset directory
```

```
os.chdir("/content/drive/MyDrive/Machine Learning project")
os.listdir()
```

```
['demo.ipynb',
 'label2type.txt',
 'load_model.py',
 'run_inference.py',
 'model.py',
 'checkpoint_best.pt',
 'checkpoint.pt',
 'final_weights.pt',
 'train (1).npz',
 'final_weights_(2)_failed.pt',
 'final_weights_date18.pt',
 'final_weights_date17.pt',
 'final_competition-description.pdf']
```

```
import torch
import random
import numpy as np

# Set a fixed random seed for reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

```
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image

class CustomDataset(Dataset):
    def __init__(self, npz_path):
        npz_data = np.load(npz_path)
        self.images = npz_data["images"] # (N, 3, 128, 128) in np.uint8
        self.labels = npz_data["labels"] # (N,) in np.int64
        assert self.images.shape[0] == self.labels.shape[0]
        print(f"{npz_path}: images shape {self.images.shape}, "
              f"labels shape {self.labels.shape}")

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        image = torch.tensor(self.images[idx]) / 255 # convert to [0, 1] range
        label = torch.tensor(self.labels[idx])
        return image, label
```

```
train_dataset = CustomDataset("train (1).npz")
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, num_workers=8, pin_memory=True, prefetch_factor=2)
```

```
train (1).npz: images shape (18900, 3, 128, 128), labels shape (18900,)
```

Start coding or [generate](#) with AI.

```
# sample data batch
images, labels = next(iter(train_loader))
print(f"images shape: {images.shape}")
print(f"labels shape: {labels.shape}")
```

```
images shape: torch.Size([128, 3, 128, 128])
labels shape: torch.Size([128])
```

```
import matplotlib.pyplot as plt

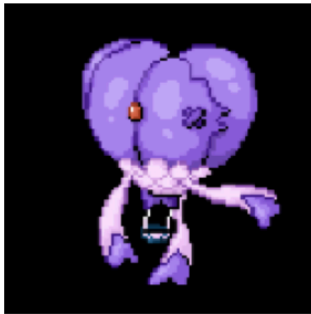
# load label text
with open('label2type.txt', 'r') as f:
    label2type = eval(f.read())

# plot samples
fig, axes = plt.subplots(1, 4, figsize=(12, 4))
for i, ax in enumerate(axes):
    label = labels[i].item()
    # (C, H, W) to (H, W, C) for plotting
    ax.imshow(images[i].numpy().transpose(1, 2, 0))
    ax.axis('off')
    ax.set_title(f"{label}: {label2type[label]}")
plt.tight_layout()
plt.show()
```

104: grass,water



12: flying,water



159: steel,water



35: dragon,flying



```
import torch
import torch.nn as nn
import torch.nn.functional as F
from tqdm import tqdm

class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size = 3, stride = 1, padding=1),
            nn.GroupNorm(min(8, channels), channels),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(channels, channels, kernel_size = 3, stride = 1, padding=1),
            nn.GroupNorm(min(8, channels), channels),
        )
        self.act = nn.LeakyReLU(0.2, inplace=True)

    def forward(self, x):
        return self.act(x + self.conv_block(x))

class ConvAE(nn.Module):
    def __init__(self, input_channels=3, latent_channels=8, num_classes=170):
        super().__init__()

        # --- Encoder ---
        self.encoder = nn.Sequential(
            nn.Conv2d(input_channels, 64, kernel_size=3, stride=2, padding=1),
            nn.GroupNorm(8, 64),
            nn.LeakyReLU(0.2, inplace=True),
```

```

        # ResBlock(64),

        nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
        nn.GroupNorm(8, 128),
        nn.LeakyReLU(0.2, inplace=True),

        nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
        nn.GroupNorm(8, 256),
        nn.LeakyReLU(0.2, inplace=True),

        # Encoder refinement
        nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
        nn.GroupNorm(8, 256),
        nn.LeakyReLU(0.2, inplace=True),
        ResBlock(256),
    )

    # --- Bottleneck linear mapping ---
    self.quant_conv = nn.Conv2d(256, latent_channels, kernel_size=1)

    # --- Decoder prep ---
    self.post_quant_conv = nn.Conv2d(latent_channels, 256, kernel_size=1)

    # --- Decoder ---
    self.decoder = nn.Sequential(

        nn.ConvTranspose2d(256, 256, kernel_size=3, stride=1, padding=1),
        nn.GroupNorm(8, 256),
        nn.LeakyReLU(0.1, inplace=True),
        ResBlock(256),

        nn.ConvTranspose2d(256, 128, kernel_size=3, stride=1, padding=1),
        nn.GroupNorm(8, 128),
        nn.LeakyReLU(0.1, inplace=True),
        nn.ConvTranspose2d(128, 64, kernel_size=3, stride=2, padding=1, output_padding=1),
        nn.GroupNorm(8, 64),
        nn.LeakyReLU(0.2, inplace=True),
        #ResBlock(64),
        nn.ConvTranspose2d(64, input_channels, kernel_size=3, stride=2, padding=1, output_padding=1),
        nn.Sigmoid()
    )

    # --- Classification head ---
    self.classifier = nn.Sequential(
        nn.AdaptiveAvgPool2d((1, 1)),
        nn.Flatten(),
        nn.Linear(latent_channels, num_classes)
    )

    def encode(self, x):
        return self.quant_conv(self.encoder(x))

    def classify(self, z):
        return self.classifier(z)

    def decode(self, z):
        return self.decoder(self.post_quant_conv(z))
#
    def forward(self, x):
        z = self.encode(x)
        x_recon = self.decode(z)
        logits = self.classify(z)
        return x_recon, z, logits

```

```

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from tqdm import tqdm
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.utils.data import random_split, DataLoader
from torch.amp import autocast, GradScaler

import torch
import torch.nn.functional as F
from tqdm import tqdm

def loss_fn(x, x_recon, logits, labels):
    recon = F.mse_loss(x_recon, x, reduction='mean')
    ce = F.cross_entropy(logits, labels)
    return torch.log(recon) + torch.log(ce), recon, ce

```

```

# — train —
def train_convae(model, train_loader, optimizer, scheduler, device, num_epochs=600):
    model.to(device)
    scaler = GradScaler()
    for epoch in range(1, num_epochs+1):
        model.train()
        sum_recon = sum_ce = sum_corr = sum_samples = 0
        loop = tqdm(train_loader, desc=f"Epoch {epoch}/{num_epochs}")
        for x, y in loop:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            with autocast(device_type='cuda'):
                x_recon, z, logits = model(x)
                loss, recon, ce = loss_fn(x, x_recon, logits, y)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

            b = x.size(0)
            sum_recon += recon.item() * b
            sum_ce += ce.item() * b
            sum_corr += (logits.argmax(1) == y).sum().item()
            sum_samples += b

            lr = optimizer.param_groups[0]['lr']
            loop.set_postfix(
                loss=f"{loss.item():.6f}",
                mse=f"{recon.item():.6f}",
                ce=f"{ce.item():.6f}",
                lr=f"{lr:.6f}"
            )

        train_recon = sum_recon / sum_samples
        train_acc = sum_corr / sum_samples
        score = train_recon / train_acc
        scheduler.step()
        lr = optimizer.param_groups[0]['lr']
        print(f"Epoch {epoch}: recon={train_recon:.6f}, acc={train_acc:.6f}, score={score:.6f}, lr={lr:.1e}")

    torch.save(model.state_dict(), 'final_weights.pt')

device = torch.device("cuda")
model = ConvAE(input_channels=3, latent_channels=8, num_classes=170).to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-3)

scheduler = CosineAnnealingLR(optimizer, T_max=200, eta_min=1e-5)
train_convae(model, train_loader, optimizer, scheduler, device, num_epochs=200)

```

```

Epoch 188/200: 100% ██████████ 148/148 [00:07<00:00, 19.56it/s, ce=3.508033, loss=-6.488503, lr=0.000020, mse=0.000000]
Epoch 188: recon=0.000354, acc=0.144180, score=0.002457, lr=1.9e-05
Epoch 189/200: 100% ██████████ 148/148 [00:07<00:00, 19.46it/s, ce=3.456148, loss=-6.650472, lr=0.000019, mse=0.000000]
Epoch 189: recon=0.000354, acc=0.145026, score=0.002441, lr=1.7e-05
Epoch 190/200: 100% ██████████ 148/148 [00:07<00:00, 19.48it/s, ce=3.450718, loss=-6.899220, lr=0.000017, mse=0.000000]
Epoch 190: recon=0.000354, acc=0.144656, score=0.002446, lr=1.6e-05
Epoch 191/200: 100% ██████████ 148/148 [00:07<00:00, 19.53it/s, ce=3.869861, loss=-6.591228, lr=0.000016, mse=0.000000]
Epoch 191: recon=0.000354, acc=0.145661, score=0.002429, lr=1.5e-05
Epoch 192/200: 100% ██████████ 148/148 [00:07<00:00, 19.51it/s, ce=3.583252, loss=-6.631584, lr=0.000015, mse=0.000000]
Epoch 192: recon=0.000354, acc=0.146455, score=0.002415, lr=1.4e-05
Epoch 193/200: 100% ██████████ 148/148 [00:07<00:00, 19.50it/s, ce=3.657285, loss=-6.679952, lr=0.000014, mse=0.000000]
Epoch 193: recon=0.000354, acc=0.146825, score=0.002408, lr=1.3e-05
Epoch 194/200: 100% ██████████ 148/148 [00:07<00:00, 19.54it/s, ce=3.638951, loss=-6.598569, lr=0.000013, mse=0.000000]
Epoch 194: recon=0.000353, acc=0.147407, score=0.002397, lr=1.2e-05
Epoch 195/200: 100% ██████████ 148/148 [00:07<00:00, 19.40it/s, ce=3.613781, loss=-6.766559, lr=0.000012, mse=0.000000]
Epoch 195: recon=0.000353, acc=0.146138, score=0.002417, lr=1.2e-05
Epoch 196/200: 100% ██████████ 148/148 [00:07<00:00, 19.40it/s, ce=3.580125, loss=-6.658890, lr=0.000012, mse=0.000000]
Epoch 196: recon=0.000353, acc=0.146243, score=0.002415, lr=1.1e-05
Epoch 197/200: 100% ██████████ 148/148 [00:07<00:00, 19.58it/s, ce=3.489223, loss=-6.680146, lr=0.000011, mse=0.000000]
Epoch 197: recon=0.000353, acc=0.147090, score=0.002401, lr=1.1e-05
Epoch 198/200: 100% ██████████ 148/148 [00:07<00:00, 19.49it/s, ce=3.245169, loss=-6.850450, lr=0.000011, mse=0.000000]
Epoch 198: recon=0.000353, acc=0.148783, score=0.002373, lr=1.0e-05
Epoch 199/200: 100% ██████████ 148/148 [00:07<00:00, 19.59it/s, ce=3.315906, loss=-6.744754, lr=0.000010, mse=0.000000]
Epoch 199: recon=0.000353, acc=0.146772, score=0.002405, lr=1.0e-05
Epoch 200/200: 100% ██████████ 148/148 [00:07<00:00, 19.39it/s, ce=3.480620, loss=-6.816707, lr=0.000010, mse=0.000000]
Epoch 200: recon=0.000353, acc=0.146296, score=0.002413, lr=1.0e-05

```

```

# ----- Visualization -----
def plot_reconstructions(model, dataloader, device, num_images=8):
    model.eval()
    with torch.no_grad():
        x = next(iter(dataloader))[0].to(device)
        x_recon, z, logits = model(x)
        x = x.cpu().numpy()
        x_recon = x_recon.cpu().numpy()
        print(f"Latent bottleneck dimension: {z.flatten(start_dim=1).shape[1]}")

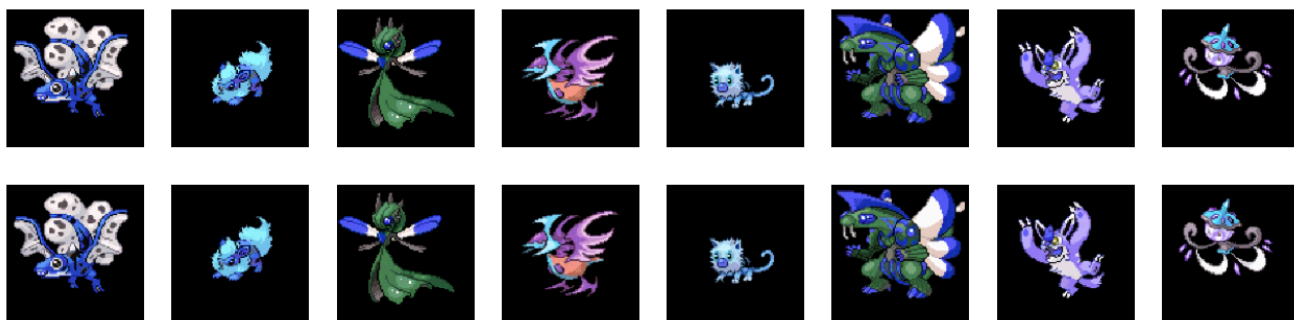
    plt.figure(figsize=(16, 4))
    for i in range(num_images):
        # Original
        plt.subplot(2, num_images, i+1)
        plt.imshow(x[i].transpose(1, 2, 0)) # (C, H, W) -> (H, W, C)
        plt.axis('off')

        # Reconstruction
        plt.subplot(2, num_images, i+1+num_images)
        plt.imshow(x_recon[i].transpose(1, 2, 0))
        plt.axis('off')
    plt.show()

plot_reconstructions(model, train_loader, device, num_images=8)

```

Latent bottleneck dimension: 8192



```

# Submission

# 1) Save model weights
torch.save(model.state_dict(), "checkpoint.pt")

# 2) Prepare the 'Model' class for submission
with open("model.py", "r") as f:
    print(f.read())

# 3) Submit the model code & weights online

import torch
import torch.nn as nn
import torch.nn.functional as F

```

```

class Model(nn.Module):
    def __init__(self, input_channels=3, latent_channels=4):
        super().__init__()
        # Make sure the layers are consistent with your checkpoint weights
        self.encoder = nn.Sequential(
            nn.Conv2d(input_channels, 64, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(128, 256, kernel_size=3, stride=1, padding=1),
            nn.ReLU()
        )
        self.quant_conv = nn.Conv2d(256, latent_channels * 2, kernel_size=1)
        self.post_quant_conv = nn.Conv2d(latent_channels, 256, kernel_size=1)
        self.decoder = nn.Sequential(
            nn.Conv2d(256, 128, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(64, input_channels, kernel_size=4, stride=2, padding=1),
            nn.Sigmoid()
        )

    def preprocess(self, x):
        # The input data is always on value range [0, 1], you may do normalization here
        return 2 * x - 1

    def encode(self, x):
        x = self.preprocess(x)
        h = self.encoder(x)
        h = self.quant_conv(h)
        mean, logvar = torch.chunk(h, 2, dim=1)
        # Make sure you only return the latent tensor, not a tuple
        return mean

    def decode(self, z):
        h = self.post_quant_conv(z)
        x_recon = self.decoder(h)
        # Make sure you only return the recon tensor, not a tuple
        return x_recon

```

```

# Metric 1: Recon MSE on test set (on value range [0, 1])
# Metric 2: Classification accuracy (linear probing with test set latents, 170 classes)
# Final Score: recon_mse / probing_accuracy (the lower the better)

```